

Applications of NNs: 1、模式识别 (Pattern Recognition/Pattern Classification) 2、回归 (Regression/Data Fitting/Function Approximation)

Mathematical Model of a Neuron

Three basic components for the model of a neuron:

- 1. A set of synapses or connecting links: characterized by a weight or strength of its own.
- 2. An adder for summing the input signals, weighted by the respective synapses of the neuron (a linear combiner).
- 3. An activation function for limiting the amplitude of the neuron output, e.g., it limits the permissible amplitude range of the output signal, typically [0, 1] or [-1, 1].

Mathematically, for a neuron k,

$$u_k = \sum_{j=1}^n w_{kj}x_j \quad y_k = \phi(u_k + b_k)$$

Where $x_1, x_2, \dots, x_n$ are the input signals; w_{kj} b_{kj} are the synaptic weights of neuron k; n is the linear combiner output due to the input signals; b_k is the bias; $\phi(\cdot)$ is the activation function, and y_k is the output signal of the neuron.

激活函数分类 Activation (Squash) Functions

阈值函数 (硬限制函数, threshold function)

$$\varphi(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

(McCulloch-Pitts) 神经元模型使用此函数

分段线性函数 (piecewise-linear function)

$$\varphi(x) = \begin{cases} 1 & x > 0.5 \\ x & -0.5 < x \leq 0.5 \\ 0 & x \leq -0.5 \end{cases}$$

S型函数 (S-shaped function, 最常用)

$$\varphi(x) = \frac{1}{1 + e^{-x}}$$

逻辑函数 (logistic function) : $\varphi(x) = \frac{1}{1 + e^{-x}}$ (a为参数)

双曲正切函数 (hyperbolic tangent function) : $\varphi(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

神经网络的连接与架构 (Connection and Architecture of Neural Networks)

- 1. 有向 (信号流) 图 (Directed (Signal-Flow) Graphs) P49
- 2. 网络架构类型 (Network Architectures)

按计算节点 (神经元) 层数可分为:

- 1. 单层前馈网络 (Single-Layer Feedforward Networks)
- 1 Input lay 2layer of output neurons
- 2. 多层前馈网络 (Multilayer Feedforward Networks,
- 1 Input lay 2layer of hidden neurons 3layer of output neurons

Artificial Neural Network

- 1. Knowledge is acquired by the network *through a learning process*.
- 2. Inter-neuron connection strengths known as synaptic weights are used to *store* the knowledge.

学习方式: 有监督(分类/回归)、无监督(聚类/PCA/SOM), 强化

L2: Perceptron

The perceptron is the first artificial neural network that can automatically adjust its parameters through learning. **做分类**

It has only **one** layer of learnable weights

The main task of a perceptron is **binary classification**

Is it possible for the perceptron to produce a decision boundary that is a nonlinear curve? **No**

The Key Question: How to **choose the proper weights**?

Two classes are **linearly separable** if and only if there exists a weight vector w based on which the perceptron can correctly perform the classification.

There **CANNOT** be any simple perceptron that achieves the classification task.

感知机的结构几何意义以及能解决线性可分

感知机 = 加权求和 + 激活函数 (阶跃函数) $v = w^T x + b$

感知机先做 “加权求和”, 然后把 v 代入 “硬阈值函数” 决策边界是 $v=0$ 即 $w^T x + b=0$

不能解决线性不可分: XOR

感知机如何学习很重要

学习目标: The perceptron’s learning goal is to make the output y match the teacher’s label d .

Learning rule: $e(n) = d(n) - y(n)$

$w(n+1) = w(n) + \eta * e(n) * x(n)$

$x(n)$: 第 n 个被 misclassify 的输入向量 $w(n)$: 第 n 次迭代时的权值向量 $d(n)$: 目标输出 (desired output) $y(n)$: 实际输出 $e(n)$: error η : 学习率 (learning rate)

怎么判断线性不可分: Just let the perceptron learn the training samples. If the weights converge, then it must be linearly separable. And if the perceptron does not stop learning, then it is not.

if samples are linearly separable, no matter the initial weights, the perceptron will eventually learn. (无论学习率也行但是影响效率)

MLP: Adding a hidden layer can transform nonlinear problems into linearly separable ones.

LMS (Least-Mean-Square

目的: 让一条直线一点一点变得更 “贴近” 数据。每看到一个样本, 就往正确方向推一下这条线。

LMS 是用来做回归的在线学习算法。目标是最小化 E(w), 误差是 e(n), 使用梯度下降推导, 得到更新规则

让线性模型去拟合真实数据 $d(i)$ 。误差 $e(i) = d(i) - y(i)$

$$y(x) = w^T x = w_1 x_1 + w_2 x_2 + \dots + w_m x_m + b$$

$$E(w) = \sum_{i=1}^n e^2(i) \quad E(n) = \frac{1}{2} e^2(n)$$

需要让代价函数 $E(w)$ 小 (上面 $E(n)$ 为瞬时代价函数), 是为不用全部样本, 只用 “当前样本” 更新一次。

用梯度下降 (Gradient Descent) 一点点调整权值

$$w(n+1) = w(n) + \Delta w(n) \quad \Delta w(n) = -\eta \nabla E(w(n))$$

$$w(n+1) = w(n) + \eta e(n)x(n)$$

因为: 1、Perceptron 的输出是 0/1, 有硬限制器, 不可做 2、LMS 输出是线性的 (没有 hard limiter) 3、 $e(n)$ 是实数, 不是 ± 1 4、梯度是可导的

所以 LMS 才能使用: 1、梯度下降 2、调整权值来拟合连续数据

L3 Multi-layer Perceptron (MLP): Backpropagation

The hidden layer *transforms* the input into a new feature space where the problem becomes linearly separable.

1. input layer 2. One or more hidden layers 3. An output layer

加权求和: $v_j = \sum w_{ij} x_i + b_j$

激活函数: $y_j = \varphi(v_j)$

adopts a **smooth nonlinear activation function**, such as sigmoid (适合二分), tanh (双曲正切), ReLU (现代常用)

反向传播算法: Backpropagation:

和 LMS 一样, 本质还是在最小化 [误差平方和]

Exactly like LMS but now we have **many layers**, so we need chain rule to propagate gradients backward.

流程: 1. Initialization 随机初始化所有权重、偏差。

2. Present training examples 每个 epoch, 按某种顺序喂入所有训练样本。

3. Forward computation 从输入层到输出层, 逐层计算 v 和 x out

4. Backward computation 在输出层计算误差, 再向后层层计算 δ ; 用 generalized delta rule 更新所有权重。

5. Iteration 若未满足停止条件 $\rightarrow$ 继续下一 epoch。

Summary,

$$w_{ij}^{(s)}(k+1) = w_{ij}^{(s)}(k) + \eta^{(s)} \delta^{(s+1)} x_{out}^{(s+1)}$$

where $\delta^{(s)} = (d_{out}^{(s)} - x_{out}^{(s)}) \phi'(v^{(s)})$ for output layer

$$\delta_j^{(s)} = \left(\sum_{k=1}^{n_{s+1}} w_{jk}^{(s+1)} \delta_k^{(s+1)} \right) \phi'(v_j^{(s)})$$

or $\delta_j^{(s)} = \left(\sum_{k=1}^{n_{s+1}} w_{jk}^{(s+1)} \delta_k^{(s+1)} \right) \phi'(v_j^{(s)})$ for hidden layers

BP is a special case of **gradient descent** method (based upon the method of **steepest descent**). The errors are propagated back from the output layer to the input layer!

BP 不会自动收敛, 所以要人为设置停止条件。4 种条件: 1. MSE 变化量够小 2. 权重变化量够小 (稳定) 3. 整个训练集 MSE 低于阈值 4. epoch 达到上限

万能逼近定理: The theorem merely states that a **single hidden layer** is sufficient for a multilayer perceptron to approximate any bounded continuous function.

MLP 能逼近连续函数吗? 可以。连续函数逼近也能逼近不连续函数 (用多阈值组合方式)

BP 的缺点: 可解释性、速度、局部最优: 1. 黑箱 (缺乏可解释性) 2. 学习慢, 常常需上 epochs 3. 只保证局部最小值, 不保证全局最优 4. 容易陷入 local minima 5. 训练依赖参数 (learning rate, hidden size)

Lecture 4: MLP: Design and Training Issues

顺序学习 vs 批量学习:

Sequential (在线学习) 每看到一个样本就更新一次优点: 噪声有助于跳出局部最小值 $\rightarrow$ 更好解。缺点: 更新 noisy, 不稳定 Batch (批量学习) 看完所有样本后才更新一次优点: 可以用二阶方法 (例如 Levenberg - Marquardt), 收敛快。缺点: 容易卡住 local minima

$$\Delta w_{ij}^{(s)}(k) = -\eta \frac{\partial E(k)}{\partial w_{ij}^{(s)}} \Delta w_{ij}^{(s)} = -\eta \frac{1}{N} \sum_{k=1}^N \frac{\partial E(k)}{\partial w_{ij}^{(s)}}$$

二阶优化 (Newton)

$$E(w(n+1)) = E(w(n)) + g^T \Delta w + \frac{1}{2} \Delta w^T H \Delta w$$

$$\Delta w(n) = -H^{-1}(n)g(n)$$

二阶方法比梯度下降快, 但要计算 Hessian $\rightarrow$ 代价高。

输入归一化 (Normalization)

$$x_i'(n) = \frac{x_i(n) - \bar{x}_i}{\sigma_i}$$

训练标准化的作用: 加快收敛, 避免某些输入主导梯度、训练更稳定

隐藏层选择

分类 sigmoid 或 softmax。回归: 线性 (purelin)

4.3 MLP 的训练技巧 (Training Considerations)

① 数据预处理 (Pre-processing)

非常重要: 所有输入必须归一化

eg: 均值归 0 (Zero mean)

方差为 1 (Unit variance)

或者映射到 [-1, 1] (or mapped to [-1, 1])

如果输入尺度差距过大, 学习会非常困难

② 什么时候用 批量学习 vs 顺序学习

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

Batch: 一次用所有样本更新

Sequential: 一次用一个小数据 subset

- 1. For a given data set containing N points $(x_i, d_i), i = 1, \dots, N$
- 2. Choose a RBF function ϕ
- 3. Calculate $\phi_j = \phi(\|x_j - x_i\|), (j, i) = 1, 2, \dots, N$
- 4. Obtain the interpolation matrix Φ
- 5. Solve the linear equation $\Phi w = d$
- 6. Get the unique solution w
- 7. Done

如何让 RBF 可用 (General RBF Networks)

$$f(x) = \sum_{i=1}^M w_i \phi(\|x - \mu_i\|) + b_i$$

其中: $M \ll N, \mu_i$: 中心, 可训练

b_i : 偏置, σ_i (宽度) 可训练

改进方式 ① 随机选中心

N 个数据点中随机选 M 个

RBF 密度统一设置: $\delta = \frac{d_{\max}}{\sqrt{2M}}$

优: 快, 缺: 要较多训练样本

② K-means 聚类

把 N 个点分为 C 个 cluster

(1) 初始化 μ_i (2) 取样本 $x(n)$

(3) 按最小距离归类 (4) 更新中心

(5) 循环直到收敛

优: $M \ll N$, 泛化更好,

RBF 中心代表数据结构

MLP & RBF 都是 supervised, SOM 是 unsupervised

Unsupervised: 1. The neural network is fed with input.

2. The weights are adjusted based upon the input signals only!

Self-Organizing Maps (SOMs)

Topological organization is the unique property of SOM.

SOM 的网络结构

输入层: n 维输入向量 $x \in R^n$

输出层: 通常是二维输出, eg: $10 \times 10, 20 \times 20$

(权重维度 = 输入而非输出)

训练流程

① Competition

$w_{ij} = \argmin \|x - w_{ij}\|$

找与输入向量最近的神经元 BMV

② Cooperation

邻域函数 $h_{ij}(x)$

离 winner 近影响大

③ Adaptation weight 朝输入方向微调

$w_j(n+1) = w_j(n) + \eta(n) h_{ij}(x) (x - w_j(n))$

学习率 邻域函数

winner 和它的邻居都要更新

训练过程中发生: 1. 初始阶段 (邻域大): 整张地图粗略成形

2. 后期 (邻域缩小): 地图微调, 边界锐化 3. 最终形成连续区域表示相似模式: 不连续边界代表不同类别/结构

使用训练好的 SOM: 对于任何新的输入 x , 找到最接近的神经元 j^* $\rightarrow$ 就知道 x 属于 SOM 的哪个区域。

SOM 学到的是: 数据之间 “相似性” 的空间布局。SOM learns a **topology-preserving projection** of input data.

L6

特点

Perceptron 线性分类器

MLP 多层 + BP

RBF 基于距离的局部模型

SOM 无监督学习

解决的问题

线性可分 快简单 不能解决 XOR

非线性映射 强灵活 训练慢 可能过拟合

分类、回归 快稳定 高维易崩

聚类、降维 可视化强 不适合大规模分类

L1 N1 Reinforcement Learning

Reinforcement learning: The goal is to maximize rewards from interaction with the environment: a sequential decision-making 顺序决策 problem.

RL 的核心: Exploration 探索 vs Exploitation 利用

MDP (马尔可夫决策过程) 是 RL 的数学模型

MDP (Markov Decision Process)

观察 State $\rightarrow$ 选 Action $\rightarrow$ reward $\rightarrow$ next state

MDP = (S, A, P, R, γ)

S: State, Action, Reward R(s, a)

P: Policy Transition probability $P(s'|s, a)$

γ : discount factor $0 \leq \gamma < 1$ 未来奖励的重要性

回报 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}(S_{t+k}, A_{t+k})$

价值函数 $V_{\pi}(s) = E[G_t | S_t = s]$

$V_{\pi}(s) = \sum_{s'} P(s'|s) q_{\pi}(s, s')$

value function action value function

$V_{\pi}(s) = \sum_{s'} P(s'|s) \pi(s'|s) V_{\pi}(s')$

$q_{\pi}(s, a) = \sum_{s'} P(s'|s, a) [R + \gamma V_{\pi}(s')]$

$V_{\pi}(s) = \sum_{s'} P(s'|s) \pi(s'|s) [R + \gamma V_{\pi}(s')]$

Bellman 转移方程 策略转移 Bellman 回报方程

新状态函数

L1 N2 Bellman Optimality Equation

Why Bellman Optimality?

找最优价值函数 V^* , q^* , 从而选最优

最优价值函数

$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R + \gamma V^*(s')]$

max over π of $V_{\pi}(s)$

$q^*(s, a) = \sum_{s'} P(s'|s, a) [R + \gamma \max_{a'} q^*(s', a')]$

因为找最优好, 不依赖任何 π

$V^*(s) = \max_a q^*(s, a)$

最优状态价值 = 最优 a 的即时奖励 + 折扣后的最优未来价值

动作 $\dots$ 下一个状态最好的动作价值

最优策略 q^* : $\pi^*(s) = \argmax_a q^*(s, a)$

在每个 s 都选择能让 $q^*(s, s')$ 最大的动作

BOM 是数学真理

P1, V1 是求解 BOM 的两种算法。

Solving MDPs: Dynamic Programming

当环境的动态模型 $(p(s', r | s, a))$ 已知时: DP 可以完全求解 MDP。DP 包括:

1. Policy Iteration (策略迭代)

2. Value Iteration (价值迭代)

Iterative Policy Evaluation, for estimating $V \approx v_{\pi}$

Input: π , the policy to be evaluated.

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in S$, and $V(\text{terminal})$ to 0

0 reward at terminal state

Loop: $\Delta \leftarrow 0$

Loop for each $s \in S$: $V(s) \leftarrow V(s)$

$V(s) \leftarrow \sum_{a \in A} \pi(a|s) \sum_{s'} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(s, |\Delta| - V(s))$

until $\Delta < \theta$

Policy Iteration (PI)

(1) 策略评估: v_{π}

$v_{\pi}(s) = \sum_{s'} \pi(s'|s) \sum_{a \in A} p(s', r |$

	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	2027	2028	2029	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	2040	2041	2042	2043	2044	2045	2046	2047	2048	2049	2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	2060	2061	2062	2063	2064	2065	2066	2067	2068	2069	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	2080	2081	2082	2083	2084	2085	2086	2087	2088	2089	2090	2091	2092	2093	2094	2095	2096	2097	2098	2099	2100	2101	2102	2103	2104	2105	2106	2107	2108	2109	2110	2111	2112	2113	2114	2115	2116	2117	2118	2119	2120	2121	2122	2123	2124	2125	2126	2127	2128	2129	2130	2131	2132	2133	2134	2135	2136	2137	2138	2139	2140	2141	2142	2143	2144	2145	2146	2147	2148	2149	2150	2151	2152	2153	2154	2155	2156	2157	2158	2159	2160	2161	2162	2163	2164	2165	2166	2167	2168	2169	2170	2171	2172	2173	2174	2175	2176	2177	2178	2179	2180	2181	2182	2183	2184	2185	2186	2187	2188	2189	2190	2191	2192	2193	2194	2195	2196	2197	2198	2199	2200	2201	2202	2203	2204	2205	2206	2207	2208	2209	2210	2211	2212	2213	2214	2215	2216	2217	2218	2219	2220	2221	2222	2223	2224	2225	2226	2227	2228	2229	2230	2231	2232	2233	2234	2235	2236	2237	2238	2239	2240	2241	2242	2243	2244	2245	2246	2247	2248	2249	2250	2251	2252	2253	2254	2255	2256	2257	2258	2259	2260	2261	2262	2263	2264	2265	2266	2267	2268	2269	2270	2271	2272	2273	2274	2275	2276	2277	2278	2279	2280	2281	2282	2283	2284	2285	2286	2287	2288	2289	2290	2291	2292	2293	2294	2295	2296	2297	2298	2299	2300	2301	2302	2303	2304	2305	2306	2307	2308	2309	2310	2311	2312	2313	2314	2315	2316	2317	2318	2319	2320	2321	2322	2323	2324	2325	2326	2327	2328	2329	2330	2331	2332	2333	2334	2335	2336	2337	2338	2339	2340	2341	2342	2343	2344	2345	2346	2347	2348	2349	2350	2351	2352	2353	2354	2355	2356	2357	2358	2359	2360	2361	2362	2363	2364	2365	2366	2367	2368	2369	2370	2371	2372	2373	2374	2375	2376	2377	2378	2379	2380	2381	2382	2383	2384	2385	2386	2387	2388	2389	2390	2391	2392	2393	2394	2395	2396	2397	2398	2399	2400	2401	2402	2403	2404	2405	2406	2407	2408	2409	2410	2411	2412	2413	2414	2415	2416	2417	2418	2419	2420	2421	2422	2423	2424	2425	2426	2427	2428	2429	2430	2431	2432	2
--	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	---